

Growth Engineer: build task

Welcome

Thank you for taking the time to apply, and congratulations on making it to the first round. We're glad you're here, and we mean that.

This is the build task tied to your application. It works like this: you build it, you deploy it, and if it holds together, we hop on a call so you can walk us through it yourself, in your own words.

Give yourself about **a day** of real, focused work. We've put a fair amount into this on purpose, because it's meant to feel like a small, honest slice of the actual job, not a trick or a speed test.

One thing we want you to know before you start: your work stays yours. We won't use it, we won't touch it, and you keep full ownership of everything you build. This exists so we can understand how you think and how you build, nothing more, and nothing less.

Take your time settling into the brief below, make yourself a coffee, and start whenever you're ready. We're rooting for you.

A bit about Velocity Growth

Velocity Growth is a global growth consultancy that combines growth strategy, market research, performance marketing, SEO, analytics, data science, pricing strategy, CRO, product-led growth, UI/UX design, and CRM systems into a single integrated engagement.

Backed by a team with growth leadership experience from Uber, Uber Eats, Accenture, Botim, Noon, and SWVL, the Velocity Growth team operates across 30+ markets spanning the GCC, Europe, UK, US, Asia Pacific and Australia.

Unlike traditional agencies that sell isolated services, Velocity operates as an embedded growth team: diagnosing bottlenecks across the entire funnel, building measurement infrastructure, and executing across acquisition, activation, and retention simultaneously.

Our methodology treats growth as a system, connecting paid media to CRM automation to product UX to analytics and pricing, rather than optimizing channels in isolation.

We serve startups (Series A through C), scale-ups, and enterprises across fintech, banking, SaaS, e-commerce, real estate, mobility, food tech, healthtech, logistics, insurance, retail, education, and travel.

Headquartered in Dubai with delivery capability across North America, Europe, and Asia-Pacific.

We run lifecycle and retention programmes for consumer brands across the Middle East and Africa, and we build our own products alongside the client work. Several client brands live inside one product, on one database, at the same time, and keeping them apart is something we promise when we sell it.

You'd be the only engineer on this build. The founders read code, but they aren't a safety net, so build it the way you'd build something you were shipping alone.

The build: a client campaign portal

Three brands' marketing teams log in and work with their own growth data. Their customer lists, campaign history, and the raw engagement log behind it are all in the data zip below.

- Kilele Rides (Kenya), the big one.
- Karoo Coaches (South Africa).
- Marrakech Express (Morocco), small, and new to us.

Each brand has two people: an owner, who can send, and an analyst, who can only look. Six logins in total. People sign in with an email and password or with their Google account, and both need to work.

What the portal should do

- Load each brand's data from the files we've given you.
- Show a contacts view, a campaigns view, and a dashboard: total customers, how many are contactable, signups per day over the last 30 days, and how each campaign performed.
- Send a campaign. The owner picks a campaign, sees exactly who it goes to and how many people that is, and confirms. The send goes out through our messaging provider. Afterwards the provider reports back what happened to each message, and your dashboard's delivery and engagement picture should follow that.
- Publish results. An owner can turn a campaign's results into a shareable link protected by a password, to send to a client who doesn't have a login.

What **done** means

Eleven things, all business rules, not nice-to-haves.

- All three brands are live and working. Six logins, each landing in its own portal, at a URL we can open, whichever way someone signs in. Owners can send, analysts can't, and someone who isn't one of the six gets in nowhere.
- A brand sees its own data and nothing from another brand. Ever, on every route into the data, including the ones added after you've moved on.
- The data loads, and the marketer can see what didn't. When something doesn't load cleanly, they see what didn't and why, without emailing you. Loading the same export twice still leaves one set of customers, not two.
- The numbers are right. A quietly wrong number is worse than no number. Where two careful people could reasonably count something two ways, say on the screen which way you counted.
- It's as usable for the big brand as the small one, even though one is about ninety times the size of another.
- Sending is safe and honest. The count on the confirmation screen is what the marketer is approving, and real money and real inboxes sit on the other side of it. A send that's interrupted or retried shouldn't send twice or half-send silently, and what someone approved last month should still read as approved.
- The provider talks back, and you keep up. Delivery, bounces, opens and unsubscribes arrive over time, including while your app isn't looking, and your picture of who's contactable has to stay correct against what actually happened.
- If someone later removes the thing that keeps the brands apart, we should find out from your tests, not from a client. Ship at least one test that would fail if that happened.
- The shared link is safe to send to a stranger. It shows one campaign's results and nothing else, and nothing a stranger could reach by guessing the address or getting past the password.
- It behaves when things go wrong. Bad input is rejected, not stored. A loading, empty, or broken screen says so. Use whatever AI tools you like, we do too, just name them. We're hiring the judgment on top of the output, so build something you can walk us through line by line.
- It's a real web app, not a prototype. It works on a phone as well as a laptop, holds together in use, and feels like something you'd put in front of a client rather than a demo you'd apologise for.

The stack

Supabase is fixed. It's what you'd own here, and it's what lets us inspect what you built using only the key you hand us. The front end can go anywhere that gives us a public URL.

Messaging provider

Your app sends through our messaging provider. Base URL:

<https://dispatcher-production-72fc.up.railway.app>. Docs, including how to authenticate, the endpoints, and the request and response formats, are at

<https://dispatcher-production-72fc.up.railway.app/v1/docs>. Your own unique API key is in the email that sent you this brief. Every request uses it.

How to submit

Everything from the standard build, via email:

- Your live URL.
- The six logins, as an email and a password for each, so we can sign in as them.
- Confirmation that Google sign-in is also live.
- Your Supabase project URL and anon key, table and function names, and which key your deployed app uses.
- A public GitHub repo with real history and a schema.sql, plus a README.
- Which AI tools you used, how long it took, and your earliest start date and notice period.

Plus, specifically for the send and the shared link:

- Where your send's progress and results are recorded, in your app or your database, so we can see what happened after the button was pressed.
- The provider key you were issued, so we can read the same delivery record you could.
- Your shared link and its password.
- A short note, 300 words max: what you tried to break before sending it, where the data-isolation guarantee lives (file and line), which number on your screens you're least sure about, and what isn't finished.

Ground rules

- Build only what's asked. We own nothing, the data is synthetic, keep it reachable, and assume the repo is public.

- UI and UX earn brownie points and are welcome, but the main thing we're grading is the data and the guarantees around it.
- With the logins you give us, we'll sign in as each user, both directly against Supabase and through your app, and make requests we expect to be turned down.
- We'll try Google sign-in with you, live, on the call.
- For the provider and the shared link: we'll read the same delivery record your app can see, and the reports your app gets back will be deliberately messy and out of order in places, the way a real provider's are. We'll press confirm on a send more than once, sometimes from two sessions at once, and we'll come at your shared link the way a stranger would.
- All of this stays inside the six test accounts and the data we've given you.

Data and access

Data:

<https://dispatcher-production-72fc.up.railway.app/data/SivlPYk5jesN2MTvMX9aEA/vg-growth-engineer-seed.zip>

SHA-256: 4961a25b151ca13ac56089ca46b94def6074c315445ec193c7bf87060683d35c

This data is the same for every candidate. Your API key is the only thing unique to you.

Questions

If anything here is unclear, just reply to the email. We'd rather answer now than have you guess and lose time on it.

Good luck. We're looking forward to seeing what you build. :)